

AUTOMATED ANALYSIS OF BUG DESCRIPTIONS
TO SUPPORT BUG REPORTING AND RESOLUTION

by

Oscar Javier Chaparro Arenas

APPROVED BY SUPERVISORY COMMITTEE:

Dr. Andrian Marcus, Chair

Dr. Yu-Chung Vincent Ng

Dr. Tien N. Nguyen

Dr. Lingming Zhang

Copyright © 2019

Oscar Javier Chaparro Arenas

All rights reserved

To my parents, for their inspiration and endless support.

To my wife, for her unconditional love and sacrifice.

PREVIEW

AUTOMATED ANALYSIS OF BUG DESCRIPTIONS
TO SUPPORT BUG REPORTING AND RESOLUTION

by

OSCAR JAVIER CHAPARRO ARENAS, BEng, MEng

DISSERTATION

Presented to the Faculty of
The University of Texas at Dallas
in Partial Fulfillment
of the Requirements
for the Degree of

DOCTOR OF PHILOSOPHY IN
SOFTWARE ENGINEERING

THE UNIVERSITY OF TEXAS AT DALLAS

August 2019

ProQuest Number:27826769

All rights reserved

INFORMATION TO ALL USERS

The quality of this reproduction is dependent on the quality of the copy submitted.

In the unlikely event that the author did not send a complete manuscript and there are missing pages, these will be noted. Also, if material had to be removed, a note will indicate the deletion.



ProQuest 27826769

Published by ProQuest LLC (2020). Copyright of the Dissertation is held by the Author.

All Rights Reserved.

This work is protected against unauthorized copying under Title 17, United States Code
Microform Edition © ProQuest LLC.

ProQuest LLC
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 - 1346

ACKNOWLEDGMENTS

This dissertation is the result of constant learning, work, dedication, and sacrifice. Many people were involved one way or another, and I am extremely grateful to them for their unconditional support and advice.

My deepest gratitude goes to my advisor, teacher, mentor, and friend, Dr. Andrian Marcus, for his invaluable lessons, guidance, and encouragement during my graduate studies. He opened my eyes about what research in software engineering is all about, and taught me how exciting and challenging academia truly is. He taught me to always keep pushing my limits, look beyond the details and the obvious, and follow a practical approach in research and teaching, and also in life. This achievement would not have been possible without his amazing feedback and mentorship. Thank you very much, Andi.

I also want to thank the current and past members of our research lab, Juan Florez, Laura Moreno, and Venera Arnaoudova, for their feedback, help, and support during all these years. Many thanks to my PhD committee members, Dr. Vincent Ng, Dr. Tien Nguyen, and Dr. Lingming Zhang, for their invaluable feedback to my work.

I would like to thank Dr. Massimiliano Di Penta, Dr. Denys Poshyvanyk, Dr. Martin Robillard, and Dr. Tom Zimmermann for taking the time and effort to write and send reference letters for me during my job search.

It was my pleasure to collaborate with extraordinary researchers from around the world. My sincere gratitude to my collaborators Jing Lu, Vincent Ng, Fiorella Zampetti, Massimiliano Di Penta, Gabriele Bavota, Carlos Bernal, Kevin Moran, Denys Poshyvanyk, and many others. My special thanks to Massimiliano Di Penta and Gabriele Bavota for their hospitality and amazing support during my visit to their research labs in Italy.

I also want to express my profound gratitude to my parents, who inspire me to continuously grow as a person; to my wife, who encourages and supports me in good and bad times; and

to my brothers, who show me different perspectives in life. Thank you all for your endless love and support. I also want to thank my friends, who have been by my side one way or another.

Many thanks to all the people involved in this dissertation and my graduate studies, including the faculty and staff members from the Computer Science department at The University of Texas at Dallas and Wayne State University.

Thank you all for making this journey one of the most enriching and rewarding experiences of my life.

June 2019

PREVIEW

AUTOMATED ANALYSIS OF BUG DESCRIPTIONS
TO SUPPORT BUG REPORTING AND RESOLUTION

Oscar Javier Chaparro Arenas, PhD
The University of Texas at Dallas, 2019

Supervising Professor: Dr. Andrian Marcus, Chair

User-written bug descriptions are the main information source for software developers to triage and fix the reported software bugs. Unfortunately, bug descriptions are often unclear, ambiguous, and miss critical information. In consequence, developers spend excessive time and effort triaging and solving the bugs, and, in many cases, they are unable to reproduce the problems, let alone fix the bugs in the code. Current bug reporting technology, which is mostly passive and does not verify the information provided by the users, provides little help in collecting high-quality bug information and improving the quality of bug descriptions.

This research aims at improving: (1) the quality of bug descriptions, and (2) the accuracy of bug resolution tasks that rely on bug descriptions. The approach towards achieving our goals is the automatic analysis of bug descriptions in combination with software analysis techniques. In the context of our goals, we focus on two bug resolution tasks, namely, automated duplicate bug report detection and bug localization in source code. In addition, we focus on analyzing the main information in bug descriptions, namely, the unexpected software behavior (*i.e.*, the *observed behavior*), the steps to reproduce such (mis)behavior (*i.e.*, the *steps to reproduce*), and the normal software behavior (*i.e.*, the *expected behavior*).

In this dissertation, we propose automatic techniques to identify, verify, and leverage the *observed behavior*, the *expected behavior*, and the *steps to reproduce* from bug descriptions, for

generating automated feedback to reporters about problems in their bug descriptions, and for improving the accuracy of bug resolution tasks. Specifically, we propose techniques and present empirical results on (i) discovering discourse patterns used by reporters to describe bugs; (ii) automatically detecting missing information in bug descriptions; (iii) automatically assessing the quality of the steps to reproduce provided in such descriptions; and (iv) leveraging bug descriptions and query reformulation to improve automated bug localization and duplicate bug report detection.

The proposed techniques aim to reduce the effort and time devoted by developers when triaging and solving software bugs. We expect that the feedback provided by these techniques will help write higher-quality bug descriptions, while facilitating the detection of duplicate bug reports and the localization of the reported bugs in source code.

TABLE OF CONTENTS

ACKNOWLEDGMENTS	v
ABSTRACT	vii
LIST OF TABLES	xiii
LIST OF FIGURES	xv
CHAPTER 1 INTRODUCTION	1
1.1 Problem Description	1
1.2 Research Goal	3
1.3 Contributions	4
1.4 Dissertation Organization	6
1.5 Bibliographical Notes	7
CHAPTER 2 BACKGROUND AND LITERATURE REVIEW	8
2.1 Bug Reporting	8
2.2 Bug Report Quality Assessment	9
2.3 Bug Report Characterization and Analysis	10
2.4 Duplicate Bug Report Detection	11
2.5 Text-Retrieval-based Bug Localization	14
2.6 Query Reformulation in Software Engineering	18
CHAPTER 3 THE DISCOURSE USED IN BUG DESCRIPTIONS	20
3.1 Introduction	20
3.2 Research Questions and Context	21
3.3 Bug Report Sampling	22
3.4 Coding Procedure	23
3.5 Coding Criteria	25
3.6 Inter-coder Agreement	26
3.7 Bug Description Information	26
3.8 Bug Descriptions' Discourse	27
3.9 Discourse Analysis across Projects	31
3.10 Threats to validity	33

3.11	Related work	33
3.12	Conclusions	34
3.13	Acknowledgments	35
CHAPTER 4 DETECTING MISSING INFORMATION IN BUG DESCRIPTIONS		36
4.1	Introduction	36
4.2	Regular Expressions-based DEMIBuD	37
4.3	Heuristics-based DEMIBuD	37
4.4	Machine Learning-based DEMIBuD	38
4.4.1	Textual Features	38
4.4.2	Learning Model	39
4.5	Empirical Evaluation	39
4.5.1	Context and Research Questions	39
4.5.2	Text Preprocessing	40
4.5.3	Tuning and Evaluation Settings	41
4.5.4	Evaluation Metrics	42
4.5.5	Results and Discussion	43
4.5.6	Threats to Validity	50
4.6	Related work	51
4.7	Conclusions	51
4.8	Acknowledgments	52
CHAPTER 5 ASSESSING THE QUALITY OF THE STEPS TO REPRODUCE IN BUG DESCRIPTIONS		53
5.1	Introduction	53
5.2	EULER: Assessing S2R quality	54
5.2.1	S2Rs Identification	55
5.2.2	Execution Model Generation	58
5.2.3	Base Matching Algorithm.	60
5.2.4	Step Resolution	62
5.2.5	Step Matching	65

5.2.6	Step Execution and Inference	67
5.2.7	Random System Exploration	67
5.2.8	Quality Report Generation	68
5.3	Empirical Evaluation	70
5.3.1	Bug Report Sample	70
5.3.2	Ideal Reproduction Scenarios	71
5.3.3	EULER Implementation and Calibration	72
5.3.4	Methodology	73
5.3.5	Metrics	74
5.3.6	Results and Analysis	75
5.3.7	Threats to Validity	83
5.4	Related Work	84
5.5	Conclusions	85
5.6	Acknowledgments	86
CHAPTER 6 USING BUG DESCRIPTIONS TO REFORMULATE QUERIES DURING BUG LOCALIZATION		87
6.1	Introduction	87
6.2	Query Reformulation Strategies	90
6.3	Empirical Evaluation Design	93
6.3.1	TRBL Techniques	94
6.3.2	TRBL Technique Implementation	95
6.3.3	TRBL Data	100
6.3.4	Low-quality Queries	103
6.3.5	Structure Identification in Bug Descriptions	106
6.3.6	Evaluation Procedure and Metrics	113
6.3.7	Analysis Framework	118
6.4	Evaluation Results and Discussion	122
6.4.1	Performance for Lucene	122
6.4.2	Performance for Lobster	125

6.4.3	Performance for BugLocator	128
6.4.4	Performance for BRTracer	132
6.4.5	Performance for Locus	135
6.4.6	Analysis across Code Granularities	137
6.4.7	Overall Performance	142
6.4.8	Discussion	147
6.4.9	Successful vs. Unsuccessful Queries	152
6.5	Threats to Validity	156
6.6	Conclusions	159
6.7	Acknowledgments	161
CHAPTER 7	QUERY REFORMULATION FOR DUPLICATE BUG REPORT DE- TECTION	162
7.1	Introduction	162
7.2	Query Reduction Approach	164
7.3	Empirical Evaluation	165
7.3.1	Duplicate Bug Report Detector	166
7.3.2	Bug Report Data Set	167
7.3.3	Queries and Ground Truth	168
7.3.4	Low-quality Queries	170
7.3.5	Observed Behavior Identification	172
7.3.6	Query Reduction Strategies and Metrics	175
7.3.7	Results and Discussion	177
7.3.8	Threats to Validity	183
7.4	Related Work	184
7.5	Conclusions	187
7.6	Acknowledgments	188
CHAPTER 8	CONCLUSIONS AND FINAL REMARKS	189
REFERENCES		193
BIOGRAPHICAL SKETCH		213
CURRICULUM VITAE		

LIST OF TABLES

3.1	Number of bug reports containing OB, EB, and S2Rs.	27
3.2	Number of patterns used to express the OB, EB, and S2Rs.	28
4.1	Number of <i>validation bug reports</i> missing the EB and S2Rs.	40
4.2	Overall <i>within-project</i> detection accuracy of DEMIBuD.	43
4.3	Overall <i>cross-project</i> accuracy of DEMIBuD-ML.	49
5.1	Quality annotations for the S2Rs in bug reports	68
5.2	Accuracy results for EULER’s Quality Annotations (QAs).	76
6.1	Performance differences of BugLocator implementations.	98
6.2	Performance differences of BRTracer implementations.	100
6.3	Statistics of the TRBL datasets.	101
6.4	Number of <i>low-quality</i> queries for N=5.	105
6.5	Number of <i>low-quality</i> queries for N={5, 10, ..., 30}.	106
6.6	Number of sampled queries for N={5, 10, ..., 30}.	108
6.7	Number of sampled queries for N=5.	108
6.8	Number of sampled queries containing TITLE, OB, EB, S2Rs, and CODE. . . .	113
6.9	Number of reduced queries for N=5.	114
6.10	Reduced queries and applicability of each reformulation strategy.	119
6.11	TRBL performance of each reformulation strategy when using Lucene.	121
6.12	Categorization of each reformulation strategy when using Lucene.	123
6.13	TRBL performance of each reformulation strategy when using Lobster.	126
6.14	Categorization of each reformulation strategy when using Lobster.	127
6.15	TRBL performance of each reformulation strategy when using BugLocator. . . .	130
6.16	Categorization of each reformulation strategy when using BugLocator.	131
6.17	TRBL performance of each reformulation strategy when using BRTracer.	133
6.18	Categorization of each reformulation strategy when using BRTracer.	134
6.19	TRBL performance of each reformulation strategy when using Locus.	136
6.20	Categorization of each reformulation strategy when using Locus.	137
6.21	TRBL performance improvement across datasets when using Lucene.	138

6.22	<i>Consistency</i> results across datasets when using Lucene.	140
6.23	TRBL performance when using all five TRBL techniques.	144
6.24	MRR/MAP results for all five TRBL techniques.	145
6.25	Categorization of each strategy when using all five TRBL techniques.	146
6.26	Proportion of Successful and Unsuccessful Queries.	155
7.1	Statistics of the duplicate detection data set.	168
7.2	Number of <i>low-quality</i> queries for $N=\{5, 10, 15, 20\}$	172
7.3	Number of sampled/coded bug reports and reduced queries.	174
7.4	Recall Rate@N results for each reformulation strategy.	178
7.5	Number of Successful and Unsuccessful Queries.	181

PREVIEW

LIST OF FIGURES

3.1	Most common OB discourse pattern.	29
3.2	Most common EB discourse pattern.	30
3.3	Most common S2R discourse pattern.	31
3.4	Distribution of pattern appearance across projects.	32
5.1	EULER’s workflow and main components.	55
5.2	Grammatical tree for the sentence: “ <i>When I create an entry for a purchase, the autocomplete list shows up</i> ”.	58
5.3	EULER’s Quality Report for Schedule #154 [42].	69
5.4	Perceived Comprehensibility and Usefulness of the Quality Reports.	81
6.1	Bug report #89621 from Eclipse.	92
6.2	Bug report #101434 from Eclipse.	111
6.3	Bug report #102778 from Eclipse and its corresponding buggy file.	148
6.4	Bug report #4330 from ArgoUML and its corresponding buggy class.	150
6.5	Bug report #727 from Math and its corresponding buggy method.	154
7.1	OB content in in OpenOffice’s bug report #104924.	176
7.2	OB content in Ambari’s bug report #4968.	176

CHAPTER 1

INTRODUCTION

1.1 Problem Description

When software does not behave as expected, users and developers report the problems using bug/issue trackers [250]. Specifically, problems are frequently reported as *bug reports*, documents that describe software bugs and are expected to contain the information needed by the developers to triage and fix the bugs in the software [249, 237]. While much of the information in bug reports is structured (*e.g.*, operating system or system version affected by the bug), the main content of a bug report is unstructured, that is, expressed in natural language [249, 98, 198]. Natural-language bug descriptions¹, produced by reporters, are meant to include the unexpected software behavior, observed by the user (*i.e.*, the Observed Behavior or OB); the steps followed by the user to reproduce the (mis)behavior (*i.e.*, the Steps to Reproduce or S2Rs); and the normal software behavior (*i.e.*, Expected Behavior or EB). These three types of information in bug descriptions are essential for developers to triage and fix the reported bugs [249].

While considered extremely useful, bug descriptions are often unclear, incomplete, and ambiguous [1, 108, 249, 248, 73, 115]. Recently, developers from more than 1.3k open-source projects wrote a letter to GitHub expressing their frustration that essential information (*e.g.*, S2Rs) is often missing in bug descriptions [1]. In addition, developers ask for a solution that would make users report high-quality information in such descriptions.

Low-quality bug descriptions negatively impact users and developers in many ways during bug triage and resolution. Low-quality bug content is one of the main reasons for non-reproducible bugs [108], unfixed bugs [248], and additional bug triage effort, for example, when checking for duplicate bug reports [73], as developers have to spend extra time

¹ In this dissertation, we use *bug reports* and *bug descriptions* interchangeably, unless otherwise noted.

and effort understanding the reported bugs or asking for clarifications and additional information [108, 73]. Low-quality bug descriptions are also likely to gain low attention by developers [115], and contain noisy information that hinders automated bug localization and duplicate bug report detection approaches [90]. As indicated by developers, low-quality information in bug descriptions is the main cause for delay on bug localization and fixing [249, 248]. Low-quality bug descriptions are also the main problem with automated approaches attempting to generate test cases and reproduce the described bugs using bug reports [131, 109, 242].

One of the main reasons for low-quality content in bug descriptions is inadequate tool support for bug reporting [1, 249, 108]. In the aforementioned GitHub petition [1], developers called for improvements to GitHub’s issue tracker to ensure that essential information is reported by the users. This problem extends to other bug/issue tracking systems, which capture unstructured bug descriptions through web forms without any content verification or enforcement. Some bug tracking systems (*e.g.*, Bugzilla in the Mozilla Firefox project [10]) provide semi-structured reporting of natural language information, using separate text fields or predefined text templates that explicitly ask for the OB, EB, and S2Rs. Such a solution is insufficient to address the problem, as it does not guarantee that reporters will provide high-quality information as expected.

Little research has been conducted to help users improve the quality of their bug descriptions. Zimmerman *et al.* [249] proposed an approach that predicts the quality level of bug reports. However, this approach does not provide actionable recommendations to reporters on how to improve their bug descriptions. Moran *et al.* [164] developed a technique to augment the steps to reproduce in bug reports via screenshots and graphical component images. Unfortunately, this approach does not focus on improving the textual content of bug descriptions. More recently, the same authors proposed an approach that finds, reproduces, and reports potential crashes in mobile applications without any user intervention [163].

However, this approach is not intended to support users when textually describing software bugs. Despite these attempts to support users on bug reporting, the fact remains that *bug descriptions are of low-quality and there is limited tool support to improve their quality.*

1.2 Research Goal

Our end goal is to support users and developers during bug reporting and resolution, by improving: (1) the quality of bug descriptions and (2) the accuracy of bug resolution tasks that rely on bug descriptions. The approach towards achieving our goal is the automatic analysis of bug descriptions in combination with software analysis techniques. In the context of our research, we focus on two bug resolution tasks, namely, duplicate bug report detection and bug localization in source code. In addition, we focus on analyzing the main information in bug descriptions, namely, the observed behavior, the expected behavior, and the steps to reproduce the bug.

Towards achieving our end goal, our research addresses the following research challenges:

- RC1. Understand how users describe software bugs in bug descriptions.
- RC2. Automatically identify the main information of bug descriptions.
- RC3. Automatically assess the quality of bug descriptions.
- RC4. Determine the most relevant information from bug descriptions with respect to bug resolution tasks.

We address the third challenge (*i.e.*, RC3.) towards improving the quality of bug descriptions, and the fourth challenge (*i.e.*, RC4.) towards improving the accuracy of bug localization and duplicate bug report detection techniques. The first two challenges (*i.e.*, RC1. and RC2.) are addressed toward determining how automated textual analysis of bug descriptions can be used to overcome the remaining two challenges.

1.3 Contributions

This dissertation makes the following research contributions:

1. *A catalog of discourse patterns that reporters use for describing bugs.* We verified the hypothesis that reporters use a well-defined set of *discourse patterns* to express the OB, EB, and S2Rs in bug descriptions. Discourse patterns are linguistic rules that capture the syntax and semantics of sentences/paragraphs that convey an OB, EB, or S2R. We verified the hypothesis by conducting a qualitative study based on open coding on nearly 1.1k bug reports from nine software projects. The study resulted in a catalog of 154 discourse patterns that reporters recurrently use to describe the OB, EB, and S2Rs in bug descriptions. From this set, only a handful of patterns (*i.e.*, 14.3% or 22) are used to describe the OB, EB, and S2Rs in most of the analyzed bug descriptions (*i.e.*, 82% on average across projects). The results indicate that bug descriptions present textual regularities that enable the automatic detection and analysis of the OB, EB, and S2Rs. This work targets the first research challenge (*i.e.*, RC1.) and is detailed in Chapter 3.
2. *An automated approach for detecting missing information in bug descriptions.* We proposed an approach (called DEMIBUD) that automatically detects missing EB and S2Rs in bug descriptions. DEMIBUD leverages the inferred discourse patterns and combines Natural Language Processing (NLP) and Machine Learning (ML) techniques. DEMIBUD is meant to alert reporters about missing EB and S2Rs, so that they can provide such information while they are writing and submitting their bug reports. We evaluated DEMIBUD on 1.8k bug reports from nine software systems to determine its detection accuracy. The results indicate that DEMIBUD, in its best setting, detects missing EB (S2R) with 85.9% (69.2%) average precision and 93.2% (83%) average recall. We also found that the best version of DEMIBUD can be deployed in new

projects without the need of being retrained, in which case, similar accuracy levels are expected. This work targets the second and third research challenges (*i.e.*, RC2. and RC3.). Chapter 4 provides further details of the approach and its evaluation.

3. *An automated approach for assessing the quality of the steps to reproduce in bug descriptions.* We proposed an approach (called EULER) that automatically analyzes a bug description, identifies the provided S2Rs, assesses the quality of each step, and generates actionable feedback to reporters about ambiguous steps, steps described with unexpected vocabulary, and steps missing in the bug description. This feedback is expected to help reporters improve their bug descriptions when they are writing and submitting them via the issue/bug tracker. EULER combines NLP, ML, dynamic system analysis, and textual matching techniques to automatically identify the S2Rs and assign to each S2R a set of quality annotations with specific feedback to the reporter. The feedback provided by EULER was assessed by external evaluators for 24 bug reports of six Android applications. The results indicate that EULER correctly identified 98% of the existing S2Rs and inferred 58% of the missing ones (with a 31% precision), while 73% of its quality annotations are correct. The evaluators agree that EULER is potentially useful in helping reporters improve their bug reports. This work targets the second and third research challenges (*i.e.*, RC2. and RC3.). Chapter 5 provides further details about EULER and its evaluation.
4. *A query reformulation approach based on bug descriptions to improve bug resolution tasks.* We proposed an approach to reformulate initial queries that fail to retrieve the relevant software documents when using Text Retrieval (TR)-based techniques for automated bug localization (*a.k.a.* TRBL) and duplicate bug report detection (*a.k.a.* TRDD). Within this approach, we proposed a set of query reduction strategies that retain the most relevant content of the bug description as a (new) query and discard

the remaining, irrelevant content with respect to TRBL and TRDD. We empirically evaluated the effectiveness of the queries reduced by the proposed strategies compared to initial queries formulated from the entire bug description, using state-of-the-art TR-based techniques on a large set of bug descriptions from different software systems. The results reveal that these strategies significantly improve the detection performance of existing techniques across multiple data sets and document granularities. This work targets the fourth research challenge (RC4.). We describe the query reformulation strategies and their evaluation in Chapters 6 and 7.

The proposed techniques aim to reduce the effort and time devoted by developers when triaging and solving software bugs. We expect that the feedback provided by these techniques will help write higher-quality bug descriptions, while facilitating the detection of duplicate bug reports and the localization of the reported bugs in source code.

1.4 Dissertation Organization

The rest of this dissertation is organized as follows. In Chapter 2, we present an overview of existing methods of bug reporting, bug report quality assessment, textual analysis of bug reports, and text-retrieval-based duplicate bug report detection and bug localization. Chapter 3 details our investigation of the discourse used by reporters to describe bugs. Later on, in Chapter 4, we introduce DEMIBUD, an automated technique that detects missing information in bug descriptions, and in Chapter 5, we present EULER, an automated approach that identifies and assesses the quality of the *steps to reproduce* provided in such descriptions. Chapters 6 and 7 specify the proposed query reduction techniques based on bug descriptions to automatically detect duplicate bug reports and locate the reported bugs in source code. Finally, in Chapter 8, we conclude by summarizing our main contributions and discussing directions of future research.

1.5 Bibliographical Notes

Parts of this dissertation were previously published in international conferences or journals, in collaboration with other researches.

Chapters 3 and 4 were published and presented at the 11th ACM Joint Meeting on the Foundations of Software Engineering (ESEC/FSE'17). The paper published in this conference [88] is the result of collaboration with Jing Lu and Dr. Vincent Ng from The University of Texas at Dallas; Fiorella Zampetti and Dr. Massimiliano Di Penta from the University of Sannio; Dr. Laura Moreno from Colorado State University; and Dr. Gabriele Bavota from Università della Svizzera italiana.

Chapter 5 was accepted for publication at the 27th ACM Joint Meeting on the Foundations of Software Engineering (ESEC/FSE'19). This work [80] is a collaborative effort with Carlos Bernal-Cárdenas, Kevin Moran, and Denys Poshyvanyk from the College of William and Mary; Jing Lu and Dr. Vincent Ng from The University of Texas at Dallas; and Dr. Massimiliano Di Penta from the University of Sannio.

Chapter 6 was published at the Empirical Software Engineering journal [85]. This publication is an extended version of the paper [83] published and presented at the 33rd IEEE International Conference on Software Maintenance and Evolution (ICSME'17). These manuscripts were published in collaboration with Juan Manuel Florez from The University of Texas at Dallas.

Chapter 7 was published and presented at the 26th IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER'19). The paper published in this conference [86] is a collaborative effort with Juan Manuel Florez and Unnati Singh from The University of Texas at Dallas.